

Security Assessment Summary

OWASP Juice Shop — Web Application Penetration Test
Cyber Security & Ethical Hacking

1. Project Overview

This report presents the findings of a controlled white-box security audit conducted against OWASP Juice Shop — an intentionally vulnerable web application developed by the Open Worldwide Application Security Project (OWASP) for educational purposes. The application simulates a realistic online store complete with user registration, product search, a shopping basket, and administrative functions, making it an ideal target for practising real-world penetration testing techniques.

This assessment was carried out as part of a Cyber Security & Ethical Hacking course assignment. Five distinct vulnerabilities spanning five OWASP Top 10 categories were discovered, exploited, and documented, along with concrete remediation guidance for each finding.

2. Objectives

- Deploy OWASP Juice Shop locally using Docker on Kali Linux.
- Perform reconnaissance using Nmap to map open services.
- Intercept and manipulate HTTP traffic using Burp Suite.
- Identify and exploit a minimum of five OWASP Top 10 vulnerabilities.
- Document reproduction steps, evidence, and remediation for every finding.
- Discover the hidden Score Board challenge page.

3. Environment & Tools

Component	Details
Operating System	Kali Linux (x64)
Runtime	Node.js v22.21.1 (detected automatically by Juice Shop)
Target Application	OWASP Juice Shop (bkimminich/juice-shop:latest via Docker)
Target URL	http://localhost:3000
Reconnaissance Tool	Nmap 7.95 — service version scan (nmap -sV localhost)
Proxy / Interception	Burp Suite — HTTP intercept, Repeater, Intruder modules
JWT Analysis	jwt.io online decoder
Hash Cracking	CrackStation (MD5 hash analysis)

4. Vulnerability Findings Summary

#	Vulnerability	OWASP	Tool Used	Severity
1	Broken Access Control — IDOR (Basket API)	A01	Burp Suite	High
2	Cryptographic Failures — Insecure JWT / MD5 Password in Token	A02	Browser DevTools + jwt.io	High
3	SQL Injection — Authentication Bypass (' OR 1=1--)	A03	Browser	Critical
4	Security Misconfiguration — Exposed /ftp Directory Listing	A05	Browser	Medium
5	Identification & Auth Failures — Account Enumeration + KBA Bypass	A07	Burp Suite	High

5. Detailed Findings

Finding 1: Broken Access Control — Insecure Direct Object Reference (IDOR)

A01 · Severity: High

Description

The basket API endpoint (/rest/basket/{id}) did not validate whether the authenticated user owned the requested basket. By intercepting the HTTP request with Burp Suite and changing the numeric basket ID from 7 to 6, User B was able to retrieve User A's basket contents — including product details and pricing — without any authorisation error.

Remediation

- Enforce server-side ownership checks on every resource request.
- Implement Role-Based Access Control (RBAC) and centralised access-control middleware.
- Replace predictable numeric IDs with indirect object references (e.g., UUIDs).

Finding 2: Cryptographic Failures — Sensitive Data in JWT Payload

A02 · Severity: High

Description

Upon login the application stores a JWT in browser localStorage. The token's payload (base64url-encoded, not encrypted) was found to contain the user's email address and an MD5-hashed password. MD5 is a broken algorithm — the hash can be instantly cracked using public rainbow-table tools such as CrackStation — effectively exposing plaintext credentials to anyone who obtains the token via XSS.

Remediation

- Never include passwords (hashed or otherwise) in JWT payloads.
- Store JWTs in HttpOnly, Secure, SameSite cookies — not localStorage.
- Use strong, modern hashing (bcrypt or Argon2) for passwords stored server-side.
- Keep JWTs short-lived and implement token rotation and revocation.

Finding 3: SQL Injection — Authentication Bypass

A03 · Severity: Critical

Description

The login form did not sanitise user input before embedding it in a SQL query. Entering ' OR 1=1-- in the email field broke the query string, appended an always-true condition, and commented out the password check, granting immediate access as the Administrator account (admin@juice-sh.op) with no valid credentials.

Remediation

- Use parameterised queries / prepared statements — never concatenate user input into SQL.
- Apply strict input validation and reject special SQL characters at the application layer.
- Restrict the database user to the minimum required privileges.

Finding 4: Security Misconfiguration — Exposed FTP Directory Listing

A05 · Severity: Medium

Description

The server's robots.txt file revealed the path /ftp. Navigating to http://localhost:3000/ftp rendered a full directory listing with no authentication required, exposing internal files such as backup files (.bak), configuration files, and a quarantine folder containing malware URL samples.

Remediation

- Disable automatic directory indexing in the web server configuration.
- Require authentication before serving any files under /ftp.
- Remove or relocate sensitive internal files from publicly accessible directories.
- Do not reference restricted paths in robots.txt.

Finding 5: Identification & Authentication Failures — Account Takeover via KBA

A07 · Severity: High

Description

Two weaknesses were chained: (1) Valid user email addresses were publicly visible in product reviews (endpoint /rest/products/reviews), enabling account enumeration. (2) The Forgot Password flow used a static Knowledge-Based Authentication (KBA) security question. Using Burp Intruder in Sniper mode, the correct answer was identified by comparing HTTP response codes (401 for wrong answers, 200 for the correct one), resulting in a full account takeover of bender@juice-sh.op.

Remediation

- Remove or mask email addresses from public-facing review endpoints.
- Replace security questions with secure, time-limited, token-based email verification.
- Implement rate limiting and account lockout on all authentication endpoints.
- Enforce Multi-Factor Authentication (MFA) for sensitive operations.

6. Skills Demonstrated

Skill Area	Activity
Reconnaissance	Nmap service/version scanning; robots.txt enumeration; application mapping via Burp Suite proxy.
Interception & Manipulation	Burp Suite Proxy, Repeater, and Intruder used to intercept, modify, and automate HTTP requests.
Access Control Testing	Manual IDOR exploitation by altering object references in authenticated API requests.
Cryptographic Analysis	JWT payload decoding; identification of weak MD5 hashing; evaluation of insecure token storage.

Injection Testing	SQL injection payload crafting to bypass authentication; understanding of query structure manipulation.
Misconfiguration Discovery	Directory traversal via exposed paths; unauthenticated file download from /ftp.
Authentication Testing	Account enumeration via public data; KBA brute-force using Burp Intruder Sniper mode.
Reporting	Structured vulnerability documentation following industry-standard format (description, steps, evidence, remediation).

7. Key Recommendations

- **Adopt Parameterised Queries:** Eliminate all SQL injection attack surfaces by using prepared statements throughout the codebase.
- **Harden JWT Implementation:** Remove sensitive fields from token payloads; switch to short-lived tokens stored in HttpOnly cookies.
- **Upgrade Password Hashing:** Replace MD5 with bcrypt or Argon2id with appropriate work factors.
- **Enforce Server-Side Authorisation:** Validate resource ownership on every API request; implement RBAC with centralised middleware.
- **Disable Directory Listing:** Configure the server to block directory indexing and protect all non-public paths behind authentication.
- **Replace Security Questions:** Implement secure email-based password reset with time-limited tokens and enforce MFA on sensitive flows.
- **Apply Rate Limiting:** Add rate limiting and temporary lockout on login, password-reset, and any other sensitive endpoints.
- **Regular Security Reviews:** Schedule recurring penetration tests and code reviews; adopt a DevSecOps pipeline with automated SAST/DAST scanning.

This assessment was conducted solely in a local, isolated environment using Docker on Kali Linux. All exploitation was performed against an intentionally vulnerable application (OWASP Juice Shop) for educational purposes only. No production systems were targeted or harmed.